ME133 Final Project: Atlas Playing Tennis

Athena Kolli, Pat Mutia, Payal Patel

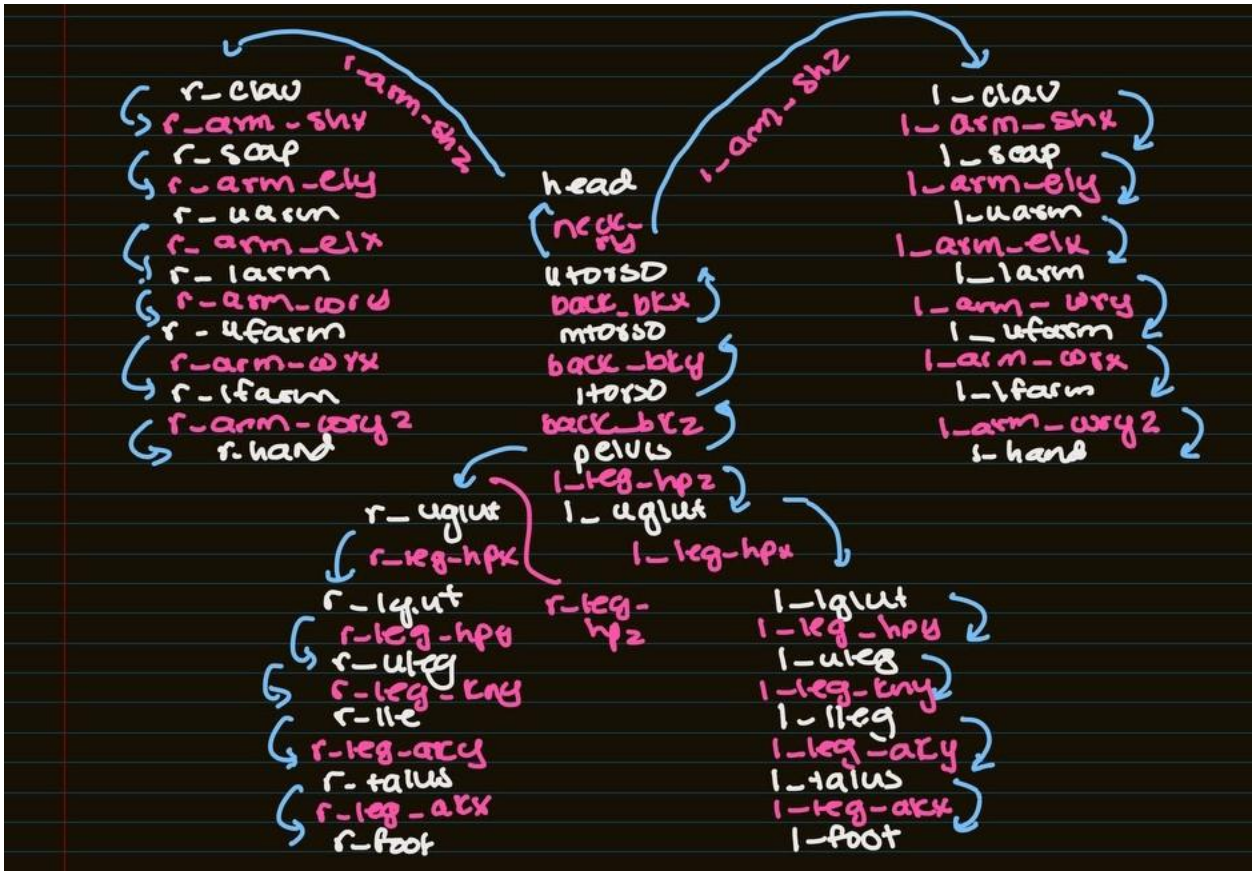
1 Introduction

The goal of this project was to simulate the humanoid robot, Atlas, playing tennis. We specifically aimed to program Atlas to hit forehands with the contact point at different positions relative to Atlas. While the feet of Atlas remain fixed on the ground, many of the other joints are able to move to help achieve hitting the ball and following the general movement that tennis players make while hitting a forehand.

Using Atlas presented some challenges as Atlas's joints do not exactly match up with human joints, so attempting to replicate the forehand of a tennis player required some creativity. Additionally, implementing the ball presented challenges, particularly at the collision. The ball was implemented by using kinematics and thus obeys gravity throughout its movement.

2 Description of System

Atlas is a 30 DOF robot. A description of the atlas joint are included below:



In order to incorporate the racket into the URDF, we included an .stl file of a racket into the "visual" field for the of the left hand link, as it did not have a physical object to visualize it. After

some work, we realized that it would also simplify our tasks to implement an additional fixed joint at the end of the left hand. We implemented this fixed joint, and placed it at the center of the racket. It is not included in the above image because fixed joints do not need to be included when building kinematic chains.



Tennis Racquet STL (found on https://www.printables.com/model/381475-tennisschlager)

After setting up the URDF, we built the kinematic chains. We established five kinematic chains in our code, all originating from the pelvis.

Kinematic Chain	Starting Link	Ending Link	Joints
Left Arm	pelvis	racket	'back_bkz', 'back_bky', 'back_bkx', 'l_arm_shz', 'l_arm_shx', 'l_arm_ely', 'l_arm_elx', 'l_arm_wry', 'l_arm_wrx', 'l_arm_wry2'
Right Arm	pelvis	r_hand	'back_bkz', 'back_bky', 'back_bkx', 'r_arm_shz', 'r_arm_shx', 'r_arm_ely', 'r_arm_elx', 'r_arm_wry', 'r_arm_wrx', 'r_arm_wry2'
Head	pelvis	head	'back_bkz', 'back_bky', 'back_bkx', 'neck_ry'
Left Leg	pelvis	l_foot	'l_leg_hpz', 'l_leg_hpx', 'l_leg_hpy', 'l_leg_kny', 'l_leg_aky', 'l_leg_akx'
Right Leg	pelvis	r_foot	'r_leg_hpz', 'r_leg_hpx', 'r_leg_hpy', 'r_leg_kny', 'r_leg_aky', 'r_leg_akx'

For this robot, we wanted to ensure that Atlas's feet stayed on the ground, and tasks for the left hand could involve the rest of the body. In order to achieve this, we needed to combine the kinematic chains. We combined the chains according to the following table:

Combined Chain	Starting Link	Ending Link
Left Arm	l_foot	racket
Right Arm	l_foot	r_hand
Right Leg	l_foot	r_foot

In order to achieve this, multiple calculations were required for each chain, to adjust the position, orientation and Jacobian. We used the following equations to do this:

Position: $${}^{f}|p_{r-f} = {}^{o}|R_f^T \ ({}^{o}|p_{r-o} - {}^{o}|p_{f-o})$$

Orientation: $${}^{f}|R_r^T = {}^{o}|R_f^T \ {}^{o}|R_r$$

Jacobian: $${}^{f}|J_r^{\omega} = {}^{o}|R_f^T \ ({}^{o}|J_r^{\omega} - {}^{o}|J_f^{\omega})$$

For each of these equations, we used the left leg position, rotation matrix, and jacobian for the terms with the f subscript. The o superscript that is shared with all the terms is our pelvis.

While the Atlas robot generally has certain bounds on the range of the joints to mimic the human body, we did not set any overarching bounds that would further limit the workspace of atlas. However, we do include secondary goals for the joints. While these secondary goals do not actually prevent Atlas from deviating from them in the case of a difficult to reach or impossible primary task, they do generally keep the robot in a prescribed position.

3 Description of Task - Atlas

The motion of atlas can be broken down into two phases, as follows:

Phase	Purpose	Time	Performed in
Phase 1	Move the center of the racket to the anticipated position of contact with the ball	0 < t < 3 s	Task Space
Phase 2	After making contact	3 <= t < 4.5 s	Joint Space

	with the ball, follow through		

In the first phase, the robot has a primary goal, which is to move the tip of the left hand, which in this case is the center of the racket, to the anticipated position of contact with the ball. There is also a secondary task to maintain the position of all joints in the robots in an “athletic” position known as ready position in tennis. The movement of the left hand drives the robot, while the rest of the joints are tasked to remain “ready” and away from singularities.

In the second phase, the movement is performed in the joint space to mimic the typical forehand follow through, which involves movements in a few of the upper arm joints and the hips. We chose to achieve this motion in the joint space because while the first phase requires an exact placement of the middle of the racket, the second phase only needs to look natural and smooth, and the exact position is not crucial.

4 Algorithm & Implementation - Atlas

To go into a bit more depth on the structure of the code for Atlas’s movement, it is set up as a series of if statements based on the current time to know which phase of the swing it is currently in. In the first phase (as described above) the position and velocity of the left arm are generated using a spline from the initial position and desired position of the center of the racket. We initially had used the goto function (which does not include starting and ending velocity) to achieve this motion, but we found that when including and tuning the velocities, we were able to get a much smoother swing. Our movement did not require a rotation, so we included a rotation of zero.

We then perform fkin on the left leg and left arm current joint positions, and use these values and the following combined kinematic chain equations to collect all the components necessary for inverse kinematics.

Position: $${}^{f}|p_{r-f} = {}^{o}|R_f^T \ ({}^{o}|p_{r-o} - {}^{o}|p_{f-o})$$

Orientation: $${}^{f}|R_r^T = {}^{o}|R_f^T \ {}^{o}|R_r$$

Jacobian: $${}^{f}|J_r^{\omega} = {}^{o}|R_f^T \ ({}^{o}|J_r^{\omega} - {}^{o}|J_f^{\omega})$$

These equations allow us to make sure all of our inputs are now in the frame of the stationary left foot. For the left leg and left arm, we create Jacobians that match the number of joints of the system, and DOFs of each joint, yielding a 6 x 30 matrix. We also follow this process for the right leg and arm, though we will not set primary goals for these chains.

On to the secondary goal for the joints, we use the following equation:

$$\dot{q}_{\text{secondary}} = \lambda_s(q_{\text{goal}} - q)$$

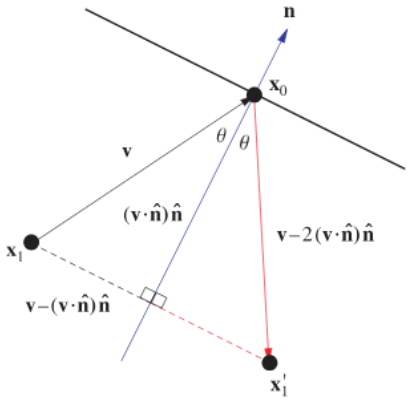
Our qgoal is a 30 x 1 matrix with the desired joint values of each of the 30 joints of the robot, and the q is stacked based on the current positions of each limb, as returned by the fkin functions for each chain. For lambda_s, we use a value of 10. Finally, we solve for q_dot using the following equation:

$$\dot{q} = J_p^+ v_p + \left(I_N - J_p^+ J_p\right) \dot{q}_{\text{secondary}}$$

In our case, the Jacobian J_p is a 6 x 30 matrix, as our only primary task is for the left arm, specifically the tip which is located at the middle of the racket. We use the Moore-Penrose inverse as our matrix is not square.
Finally, we calculate the next set of joint positions using the following equation:
q = qlast + dt*qdot

In its second phase, the approach is much simpler. We simply use a spline to go from the current joint positions to the follow through joint positions. We then form the qdot 30 x 1 matrix, setting all of the values not changed in the spline to be zero. Finally, we perform the last step to increment q: q = qlast + dt*qdot.

5 Description of Task - Ball
To create the tennis ball to simulate Atlas playing tennis, we applied simple kinematics on the ball in order to simulate a ball bouncing due to gravity ($\Delta x = v_i t + \frac{1}{2}at^2$). Then we applied the concept of reflection of a vector against a surface for elastic collision that occurs when the ball hits the center of the racquet.



6 Algorithm & Implementation - Ball
Referencing the balldemo.py code, the ball was initialized with a MarkerArray with a single id to alter aspects of the ball (i.e. position, velocity). Moreover, the ball was created within the same

node as Atlas in order for motions of the ball bouncing and Atlas performing their swings concurrently. To prevent singularities, we experimented on where the ball can bounce to in order for the racquet to properly hit the ball, which was when the final position of the ball was 0 for x, from 0 to 2 for y, and from -0.35 to 1.65 for z. Feeding in an initial velocity and acceleration due to gravity, we calculated what the initial position of the ball should be in order to produce a randomized final position within the range viable for the racquet to hit.

7 Collision Handling & Other Features

To register the racquet-ball collision, we checked where the position of the ball relative to the racquet was a small value. When the ball was within the a small specified range, the velocity of the ball was changed to perform an attractive force on the ball to reposition the ball at the center of the racquet during the Atlas's back swing. When following through with the racquet with the ball at the center, the reflected velocity against the configuration of the racquet was calculated to return to the side of the field where the ball was initialized. Then the ball was reinitialized with a new initial position to achieve a randomized final position within the hitting range of the racquet.